

LearningCpp: C++ Moderno y Arquitectura de Sistemas

De Cero Absoluto a C++ Moderno — Fundamentos de Grado Profesional (C++17 Base · Evolución C++20)

Estándar: C++17 Base (Evolución C++20) **Alcance:** 15 Módulos · 6 Fases · 117 Lecciones

Dedicación: 16 Sem. Regular / 8 Sem. Intensiva **Autor:** Jesus Vera V. (MiniLux0)

1. FICHA TÉCNICA Y FUNDAMENTACIÓN ACADÉMICA

El programa **LearningCpp** constituye un itinerario formativo riguroso orientado a la enseñanza de C++ Moderno desde cero absoluto hasta los fundamentos de grado profesional de la ingeniería de software. El curso comprende el modelo físico de la memoria (Stack, Heap, direcciones y tabla virtual) blindando el software mediante gestión determinista de recursos (RAII con `std::unique_ptr`), inmutabilidad por defecto y computación declarativa con algoritmos STL.

PARÁMETRO	ESPECIFICACIÓN TÉCNICA
Toolchain Oficial	<code>g++ ≥ 13.0</code> (MSYS2 UCRT64 / GNU Linux) o <code>Clang++ ≥ 16.0</code> (Apple Clang / LLVM).
Flags de Compilación	<code>-std=c++17 -Wall -Wextra -Wpedantic -Wconversion -Wshadow -O2</code>
Carga Horaria Sugerida	Ruta Regular: 16 semanas (6-8 h/semana) · Ruta Bootcamp: 8 semanas (12-15 h/semana). Total: ~120 horas lectivas y prácticas.
Metodología Pedagógica	<i>Break-First, Fix-Later</i> , Modelos Mentales de RAM, Principio de la Escalera, Scaffolding & Fading.
Evaluación & Entregables	15 Mini-Proyectos Modulares, 45 Demos de Bugs intencionales y 1 Proyecto Capstone Final Integrador.

2. RESULTADOS DE APRENDIZAJE Y COMPETENCIAS (RA)

RA1 · Modelo Físico de Memoria RAM

Diagnosticar la distribución física de bytes en memoria, ciclos de vida en el Stack frente al Heap y trazado exacto de direcciones hexadecimales (`0x...`).

RA2 · Gestión Determinista de Recursos (RAII)

Dominar la propiedad de memoria en el Heap con `std::unique_ptr`, erradicando por completo fugas de memoria (Memory Leaks) y accesos Use-After-Free.

RA3 · POO Segura y Polimorfismo Dinámico

Diseñar invariantes encapsuladas con constructores seguros, despacho dinámico mediante VTable, destructores virtuales obligatorios y blindaje contra Object Slicing.

RA4 · Resiliencia y Tolerancia a Fallos

Implementar políticas de excepciones basadas en Stack Unwinding, optimización de relocalización con `noexcept` y ausencias esperadas con `std::optional`.

RA5 · Metaprogramación y Programación Genérica

Construir plantillas monomórficas evaluadas en compilación con cero costo en runtime, buffers estáticos (NTTP) y funciones anónimas (Lambdas).

RA6 · Computación Declarativa y Algoritmos STL

Utilizar algoritmos estándar de la STL (`<algorithm>`) para transformaciones, búsquedas y filtros, combinándolos con `range-for` idiomáticos para secuencias directas.

3. CARGA HORARIA Y RÚBRICAS DE DOMINIO POR NIVELES

🕒 Distribución de Horas por Fase

- Fase 1 (Fundamentos): ~15h · Fase 2 (Funciones): ~12h · Fase 3 (Colecciones): ~18h
- Fase 4 (Memoria Real): ~20h · Fase 5 (POO Moderna): ~25h · Fase 6 (Resiliencia & STL): ~30h

🗺️ Rutas de Aprendizaje

- **Semestral:** 16 semanas dedicando 6 a 8 horas semanales.
- **Intensiva:** 8 semanas con dedicación inmersiva de 12 a 15 horas semanales.

Nivel Aprendiz (Fases 1-2)

Streams seguros, prevención de narrowing con `{}`, casting seguro con `static_cast` y aislamiento de Stack frames.

Nivel Intermedio (Fases 3-4)

Dominio de `std::vector` con `.at()`, paso `const &` Zero-Copy, Heap y ownership con `std::unique_ptr` (0 leaks).

Nivel Avanzado (Fases 5-6)

Jerarquías polimórficas VTable, destructores virtuales, Stack Unwinding, templates `if constexpr` y pipelines STL.

4. LOS CUATRO PILARES PEDAGÓGICOS

1. Escalera Progresiva

Cada lección se apoya estrictamente en lo visto antes.

2. Break-First, Fix-Later

Detonación de bugs reales (Undefined Behavior, Memory

3. Modelos de RAM

Diagramas visuales de hardware: Stack, Heap, punteros `0x...` y VTable de polimorfismo.

4. Scaffolding & Fading

Analogías iniciales que desvanecen de inmediato hacia la

Cero saltos mágicos o cajas negras.	Leaks) antes de aprender la solución idiomática.		terminología profesional de la industria.
-------------------------------------	--	--	---

5. MATRIZ GENERAL DE FASES Y MÓDULOS

FASE	#	MÓDULO	LECS.	EJE CONCEPTUAL Y TECNOLÓGICO	PROYECTO ENTREGABLE
Fase 1: Fundamentos	01	Getting Started	7	Compilador <code>g++</code> , flujo I/O, namespaces y veto a malos hábitos.	Terminal Interactiva
	02	Fundamental Types	7	Tipado estático, <code>sizeof</code> , <code>{}</code> uniforme, <code>static_cast</code> .	Split the Bill Calculator
	03	Scope & Control Flow	8	Control de flujo, marcos de Stack, shadowing y prevención de UB.	Taberna RPG / Cajero de UB.
Fase 2: Funciones	04	Functions	8	Pass-by-value, aislamiento de pila, RNG moderno con <code><random></code> .	Generador RPG con RNG
	05	Constants & Strings	6	<code>const</code> , <code>constexpr</code> , <code>std::string</code> y <code>std::string_view</code> .	Generador de Claves
Fase 3: Colecciones	06	Arrays & Vectors	9	Buffer Overflow de C, <code>std::vector</code> , <code>.at()</code> , multi-archivo.	Registro de Calificaciones
	07	Compound Types	7	<code>struct</code> , <code>enum class</code> , Agregados C++17 y puente a C++20.	Bestiario RPG V1
Fase 4: Memoria Real	08	References & Addresses	8	Direcciones <code>&</code> , paso por <code>const &</code> (Zero-Copy), amnesia de <code>auto</code> .	Bestiario V2 (Zero-Copy)
	09	Dynamic Memory & RAII	9	Heap, punteros observadores, <code>std::unique_ptr</code> , <code>std::move</code> , RAII.	Bestiario V3 (Heap RAII)
Fase 5: POO Moderna	10	Classes & Encapsulation	10	Encapsulamiento <code>m_</code> , <code>const</code> methods, Member Init List, <code>operator<<</code> .	Bestiario V4 (Multi-Archivo)
	11	Inheritance	7	Relación IS-A, constructores, destrucción y trampa del Object Slicing.	Jerarquía del Bestiario
	12	Polymorphism	8	Despacho dinámico, <code>virtual</code> , VTable, destructor virtual, <code>override</code> .	El Coliseo (Game Loop)
Fase 6: Resiliencia & STL	13	Error Handling & Resilience	7	Stack Unwinding, jerarquía <code>std::exception</code> , <code>std::optional</code> , <code>noexcept</code> .	Motor Resiliente
	14	Templates & Lambdas	8	Polimorfismo estático, templates en <code>.hpp</code> , NTTP, lambdas modernas.	Pipeline de Eventos
	15	STL Algorithms & Capstone	8	Algoritmos STL, Erase-Remove Idiom, iteradores, asincronía y Capstone Final.	EL MOTOR RPG DEFINITIVO

FASE 1: FUNDAMENTOS DE LA COMPUTACIÓN

MÓDULOS 01 A 03 · 22 LECCIONES

Módulo 01 — Getting Started & Pipeline del Compilador7 Lecciones · 2 Bug Demos

Objetivo: Comprender la traducción de código a binario ejecutable, dominar la E/S en consola y erradicar vicios globales como `using namespace std;`.

L00	¿Qué es programar?	Concepto de compilación y necesidad de código máquina binario.
L01	Herramientas y Terminal	Verificación de compilador <code>g++</code> y comandos de navegación básica.
L02	Primer programa (Hello World)	Anatomía de <code>int main()</code> , flujo <code>std::cout</code> y comando de compilación.
L03	Namespaces y el ámbito <code>std::</code>	Prevención de colisión de identificadores y veto a <code>using namespace std;</code> .
L04	Formato y salto de línea	Salto <code>' '</code> directo vs el sobrecosto de flush innecesario de <code>std::endl</code> .
L05	Lectura y variables iniciales	Flujo <code>std::cin</code> e inicialización uniforme <code>int x{0};</code> contra basura en RAM.
L06	Mini-Proyecto Integrador	Terminal Interactiva: lectura de líneas con <code>std::getline</code> y saneamiento.

Bug Demos: D04_Uninitialized, D05_CinSpacesModelo RAM: Pipeline & RegistrosDecisión: Veto a `using namespace std;`

Módulo 02 — Fundamental Types & Sistema de Tipado7 Lecciones · 3 Bug Demos

Objetivo: Conocer la huella física en memoria de tipos primitivos y aplicar conversión explícita con `static_cast`.

L01	Tipos primitivos y RAM	<code>int</code> , <code>double</code> , <code>char</code> , <code>bool</code> y tamaño en bytes con <code>sizeof</code> .
L02	Inicialización Uniforme <code>{}</code>	Prevención de conversiones estrechas destructivas (Narrowing Conversions).
L03	Operadores aritméticos	Precedencia y corrección de la división entera truncada ($7 / 2 = 3$).
L04	Operadores lógicos y booleanos	Compuertas booleanas y prevención del error <code>if (x = 5)</code> .
L05	Casting explícito (<code>static_cast</code>)	Transformación de tipos segura y verificada en tiempo de compilación.
L06	Deducción de tipos con <code>auto</code>	Uso idiomático de <code>auto</code> sin perder claridad sobre el estado de la RAM.
L07	Mini-Proyecto Integrador	"Split the Bill": Calculadora de propinas y división exacta de gastos.

Bug Demos: D02_Narrowing, D03_IntDivision, D04_AssignInIfModelo RAM: Bytes en Stack (sizeof)Decisión: Inicialización `{}` obligatoria

Módulo 03 — Scope & Control Flow (Control de Flujo)8 Lecciones · 4 Bug Demos

Objetivo: Dominar bifurcaciones lógicas, iteraciones y la gestión de marcos de ámbito en el Stack.

L01	Condicionales if / else	Bifurcaciones y prevención del punto y coma asesino <code>if(cond);</code> .
L02	Cadenas else if	Evaluación por cortocircuito y prevención de código inalcanzable.
L03	Bloques y Ámbito (Scope)	Ciclos de vida en Stack y prevención del bug de Variable Shadowing.
L04	Conmutación con switch	Selección múltiple y prevención del error de Fallthrough involuntario.
L05	Bucles while / do-while	Repetición condicional y prevención de bucles infinitos en consola.
L06	Bucle contador for	Inicialización, condición, incremento y prevención de errores Off-By-One.
L07	Break y Continue	Manipulación de saltos y limitaciones en bucles anidados.
L08	Mini-Proyecto Integrador	"Taberna RPG / Cajero": Menú interactivo con <code>while(true)</code> y <code>switch</code> .

Bug Demos: D01_SemicolonIf, D03_Shadowing, D04_FallthroughModelo RAM: Marcos de Ámbito {}Decisión: Scope explícito con llaves

FASE 2: FUNCIONES Y TEXTOSMÓDULOS 04 Y 05 · 14 LECCIONES

Módulo 04 — Functions & Generación de Números Aleatorios8 Lecciones · 3 Bug Demos

Objetivo: Modularizar software en unidades funcionales atómicas y dominar el aislamiento de memoria en la pila.

L01	Anatomía de una función	Firma, parámetros formales y separación de responsabilidades.
L02	Retorno de valores	Obligatoriedad de <code>return</code> en todos los caminos lógicos contra UB.
L03	Funciones void	Procedimientos sin retorno y comportamiento del compilador.
L04	Parámetros por valor	La trampa del clon: demostración de copias aisladas en la pila.
L05	Aislamiento en Stack	Independencia de memoria entre marcos de función y el <code>main</code> .
L06	Refactorización modular	Descomposición de funciones monolíticas en unidades reutilizables.
L07	Aleatoriedad con <random>	Uso de <code>std::mt19937</code> y prohibición definitiva del arcaico <code>rand()</code> .
L08	Mini-Proyecto Integrador	Generador de Atributos RPG: Tiradas probabilísticas y lógica modular.

Bug Demos: D02_MissingReturn, D04_PassByValue, D07_StaticRngModelo RAM: Call Stack FramesDecisión: Veto a `rand()`; uso de `<random>`

Módulo 05 — Constants, std::string & std::string_view6 Lecciones · 3 Bug Demos

Objetivo: Implementar inmutabilidad estricta y manipular cadenas sin sobrecosto de memoria.

L01	Constantes con const	Inmutabilidad en tiempo de ejecución (Const Correctness).
L02	Evaluación con constexpr	Cálculos evaluados por el compilador con costo cero en runtime.
L03	Cadenas std::string	Gestión de memoria dinámica interna y superación de <code>char[]</code> .
L04	Vistas std::string_view	Paso de texto Zero-Copy y prevención de Dangling Views.
L05	Saneamiento de buffers I/O	Limpieza de errores en flujos de entrada con <code>cin.clear()</code> y <code>cin.ignore()</code> .
L06	Mini-Proyecto Integrador	Generador de Contraseñas: Manipulación inmutable y validación estricta.

Bug Demos: D01_Mutation, D04_DanglingStringView, D05_CinLoopModelo RAM: Buffer de cadenas & VistasDecisión: Separación `const` vs `constexpr`

FASE 3: COLECCIONES Y ENTIDADES

MÓDULOS 06 Y 07 · 16 LECCIONES

Módulo 06 — Arrays, std::vector & Arquitectura Multi-Archivo

9 Lecciones · 3 Bug Demos

Objetivo: Gestionar colecciones dinámicas contiguas y estructurar proyectos profesionales en `.h` y `.cpp`.

L01	Límites de variables sueltas	Colapso de software ante conjuntos extensos de datos homogéneos.
L02	C-Arrays y Buffer Overflow	Demostración física de corrupción de memoria adyacente en C clásico.
L03	std::vector moderno	Inicialización con llaves <code>{5}</code> (1 elemento) vs paréntesis <code>(5)</code> (5 casillas).
L04	Acceso seguro con <code>.at()</code>	Verificación de límites obligatoria frente a la trampa de <code>[]</code> .
L05	Captura de <code>std::out_of_range</code>	Manejo táctico de excepciones con bloques <code>try / catch</code> .
L06	Range-based for	Iteración idiomática segura sin manipulación de índices manuales.
L07	Métodos de <code>std::vector</code>	<code>push_back</code> , <code>size</code> , <code>empty</code> , <code>reserve</code> y realocación en Heap.
L08	Arquitectura Multi-Archivo	Separación en <code>.h</code> y <code>.cpp</code> con directivas <code>#pragma once</code> .
L09	Mini-Proyecto Integrador	Registro de Calificaciones: Sistema multi-archivo con notas dinámicas.

Bug Demos: `D02_BufferOverflow`, `D03_BraceInit`, `D04_SilentBounds`**Modelo RAM:** Memoria Contigua en Heap**Decisión:** Uso estricto de `.at()` formativo

Módulo 07 — Compound Types (struct & enum class)

7 Lecciones · 3 Bug Demos

Objetivo: Modelar entidades compuestas y máquinas de estado con tipado estático fuerte y constructores seguros.

L01	Colapso de firmas extensas	Por qué pasar 6 parámetros individuales destruye el diseño modular.
L02	Estructuras con struct	Agrupación heterogénea y el operador de acceso punto <code>(.)</code> .
L03	Agregados y Constructores C++17	Inicialización uniforme <code>{}</code> y puente a Designated Initializers de C++20.
L04	Peligro de Números Mágicos	Fragilidad y vulnerabilidad de modelar estados con enteros.
L05	Estados con enum class	Enumeraciones fuertemente tipadas con ámbito para eliminar bugs.
L06	Colecciones de Entidades	Composición armónica de <code>std::vector<struct></code> en memoria contigua.
L07	Mini-Proyecto Integrador	Bestiario RPG V1: Base de datos en memoria con monstruos y combate.

Bug Demos: `D02_MissingSemicolon`, `D03_AggregateOrder`, `D04_MagicNum`**Modelo RAM:** Struct Alignment & Padding**Decisión:** Veto a `enum` de C; usar `enum class`

FASE 4: ARQUITECTURA DE MEMORIA REAL

MÓDULOS 08 Y 09 · 17 LECCIONES

Módulo 08 — References, Addresses & Zero-Copy

8 Lecciones · 3 Bug Demos

Objetivo: Inspeccionar direcciones de hardware y aplicar la Regla de Oro del paso por referencia constante.

L01	Costo de la copia profunda	Medición de degradación de rendimiento por clonación en memoria.
L02	Direcciones en RAM (&)	Inspección de punteros físicos hexadecimales (<code>0x...</code>).
L03	Pass-by-Reference (&)	Creación de alias directos a la variable original para mutación eficiente.
L04	Mutación accidental	Riesgo de corromper datos cuando solo se deseaba lectura.
L05	La Regla de Oro: <code>const &</code>	Tipos primitivos por valor, tipos pesados por <code>const &</code> .
L06	Amnesia de auto	Por qué <code>auto x = ref</code> descarta la referencia forzando <code>const auto&</code> .
L07	Dangling References	Retorno mortal de referencias a variables locales del Stack.
L08	Mini-Proyecto Integrador	Bestiario V2 (Zero-Copy): Refactorización integral con paso por <code>const &</code> .

Bug Demos: `D01_HeavyClone`, `D04_AccidentalMutation`, `D07_DanglingRef`**Modelo RAM:** Direcciones Hexadecimales &**Decisión:** Regla de Oro: Primitivos valor, pesados `const &`

Módulo 09 — Dynamic Memory, Smart Pointers & RAII

9 Lecciones · 3 Bug Demos

Objetivo: Comprender la asignación en el Heap y aplicar la gestión automática determinista de recursos con RAII.

L01	Stack vs Heap	Tamaño de segmentos de memoria y necesidad de memoria dinámica.
L02	Punteros crudos (T*)	Variables de dirección, operador <code>*</code> , flecha <code>→</code> y uso como observadores.
L03	Puntero nulo y Segfault	Comprobación previa defensiva y prevención de accesos inválidos.
L04	Asignación manual con new	Solicitud de memoria y por qué <code>new[]</code> está prohibido en C++ moderno.
L05	Memory Leaks	La pérdida silenciosa de recursos de RAM por omisión de <code>delete</code> .
L06	Dangling Pointers	Uso de punteros tras liberación (Use-After-Free).
L07	RAII y std::unique_ptr	Gestión determinista atando el Heap al Stack con <code>std::make_unique<T>()</code> .
L08	Semántica de Movimiento	Transferencia de propiedad con <code>std::move</code> y type aliases con <code>using</code> .
L09	Mini-Proyecto Integrador	Bestiario V3 (Heap RAII): Entidades dinámicas gobernadas por <code>unique_ptr</code> .

Bug Demos: D03_NullCrash ,
D05_MemoryLeak ,
D06_DanglingPointer**Modelo RAM:** Stack-to-Heap Ownership Pointer**Decisión:** Veto a `new[]/delete[]`; adopción estricta de RAII**FASE 5: PROGRAMACIÓN ORIENTADA A OBJETOS MODERNA**

MÓDULOS 10 A 12 · 25 LECCIONES

Módulo 10 — Classes, Encapsulation & Sobrecarga de Operadores

10 Lecciones · 3 Bug Demos

Objetivo: Blindar invariantes de software mediante encapsulamiento estricto y sobrecarga idiomática.

L01	El Estado Inconsistente	Por qué los <code>struct</code> públicos permiten corromper invariantes lógicas.
L02	El cerrojo de class	Privacidad por defecto, modificadores <code>private/public</code> y convención <code>m_</code> .
L03	Métodos const correctness	Permitir invocaciones desde referencias constantes (<code>const &</code>).
L04	Getters y Setters seguros	Retorno por <code>const &</code> y validación rigurosa de rangos en mutaciones.
L05	Member Initializer List	Inicialización en nacimiento, orden <code>-Wreorder</code> y calificador <code>explicit</code> .
L06	Tell, Don't Ask	Evitar que los getters excesivos destruyan el diseño OO.
L07	Sobrecarga operator<<	Integración natural con streams e igualdad con <code>operator==</code> .
L08	Clases Multi-Archivo	Declaración en <code>.h</code> e implementación con <code>Clase::</code> en <code>.cpp</code> .
L09	Destructores y RAII	Limpieza determinista de recursos al expirar el ciclo del objeto.
L10	Mini-Proyecto Integrador	Bestiario V4: Clases POO robustas con operadores sobrecargados.

Bug Demos: D01_InconsistentState ,
D03_ConstMember , D05_InitOrder**Modelo RAM:** Layout de Objetos en Memoria**Decisión:** Atributos privados con `m_` y Member Init List**Módulo 11 — Inheritance & Prevención de Object Slicing**

7 Lecciones · 3 Bug Demos

Objetivo: Reutilizar lógica mediante jerarquías IS-A y prevenir la destrucción de datos por Object Slicing.

L01	Anti-patrón Copiar/Pegar	Costo de duplicar lógica en entidades hermanas y necesidad de jerarquías.
L02	Herencia Simple (: public)	Relación IS-A y prevención de herencia privada accidental.
L03	Modificador protected	Atributos protegidos con métodos de acceso controlados.
L04	Cadenas de constructores	Delegación obligatoria hacia el constructor base.
L05	Ciclo de vida en jerarquías	Construcción Padre→Hijo y destrucción inversa Hijo→Padre.
L06	La trampa de Object Slicing	Destrucción de datos al guardar derivadas por valor en <code>vector<Base></code> .
L07	Mini-Proyecto Integrador	Jerarquía del Bestiario: Árbol de 3 niveles (Entidad → Monstruo → Jefe).

Bug Demos: D02b_PrivateInherit ,
D04_ConstructorChain ,
D06_ObjectSlicing**Modelo RAM:** Object Slicing en Pila**Decisión:** Herencia simple; veto a herencia múltiple con estado

Módulo 12 — Polymorphism, VTable & Despacho Dinámico		8 Lecciones · 3 Bug Demos
Objetivo: Implementar despacho dinámico en runtime con tablas virtuales y contratos de interfaz.		
L01	Early Binding (Enlace Estático)	Por qué el compilador enlaza al tipo del puntero y no al objeto real.
L02	Métodos virtual y la VTable	Despacho dinámico a través del puntero de tabla virtual <code>__vptr</code> .
L03	override y final	Detección de errores de firma con <code>override</code> y sellado de clases con <code>final</code> .
L04	El Destructor Virtual	Fuga de memoria letal al destruir polimórficamente sin <code>virtual ~Base()</code> .
L05	Interfaces Puras (= 0)	Contratos abstractos puros y herencia múltiple de interfaces.
L06	Colecciones Polimórficas	<code>vector<unique_ptr<Base>></code> y downcasting seguro con <code>dynamic_cast</code> .
L07	Impresión Polimórfica	<code>operator<<</code> delegando en método virtual puro <code>imprimir() const</code> .
L08	Mini-Proyecto Integrador	El Coliseo: Game Loop de combate polimórfico sin bifurcaciones <code>if/else</code> .
Bug Demos: <code>D01_StaticBinding</code> , <code>D03_SilentTypo</code> , <code>D04_VirtualDestructorLeak</code>		Modelo RAM: VTable & <code>__vptr</code> Decisión: Destructor virtual obligatorio en clases base

FASE 6: RESILIENCIA, ESPECIALIZACIÓN Y CAPSTONE FINAL

MÓDULOS 13 A 15 · 23 LECCIONES

Módulo 13 — Error Handling, Stack Unwinding & std::optional		7 Lecciones · 3 Bug Demos
Objetivo: Construir software tolerante a fallos con Stack Unwinding, excepciones de dominio y <code>std::optional</code> .		
L01	Fragilidad de Códigos de Retorno	Por qué retornar enteros causa errores ignorados que corrompen el sistema.
L02	Stack Unwinding & RAII	Destrucción automática de objetos RAII durante el vuelo de una excepción.
L03	Jerarquía std::exception	Captura obligatoria por referencia constante (<code>catch (const std::exception& e)</code>).
L04	Excepciones en Constructores	Abortar la instanciación de objetos zombi en estado inválido.
L05	Alternativa ligera std::optional	Manejo idiomático de ausencias esperadas con <code>std::nullopt</code> (C++17).
L06	Garantía de noexcept	Optimización crítica de relocalización de memoria en <code>std::vector</code> .
L07	Mini-Proyecto Integrador	Motor de Mazmorras: Carga de mapas con recuperación ante corrupción.
Bug Demos: <code>D01_IgnoredReturn</code> , <code>D02_RawLeakOnThrow</code> , <code>D06_VectorCopyFallback</code>		Modelo RAM: Stack Unwinding & Exception Frame Decisión: Captura por <code>const std::exception&</code> ; usar <code>std::optional</code>

Módulo 14 — Templates & Metaprogramación Genérica		8 Lecciones · 3 Bug Demos
Objetivo: Escribir código genérico monomórfico evaluado en compilación y dominar funciones anónimas Lambdas.		
L01	Fábrica de Código (Templates)	Polimorfismo estático vs dinámico: cero sobrecarga en runtime (Inlining).
L02	Plantillas de funciones	Deducción de tipos automática y plantillas multiparámetro.
L03	La trampa del Linker	Por qué las plantillas deben residir obligatoriamente en cabeceras (<code>.hpp</code>).
L04	Plantillas de clases y CTAD	Contenedores genéricos con deducción automática de tipos en C++17.
L05	Parámetros No-Tipo (NTTP)	Buffers estáticos contiguos en Stack sin tocar el Heap (<code>std::size_t N</code>).
L06	Lambdas Modernas [>(){}]	Funciones anónimas instantáneas como predicados de primer nivel.
L07	Capturas en Lambdas	Copia <code>[=]</code> vs referencia <code>[&]</code> y prevención de Dangling Captures.
L08	Mini-Proyecto Integrador	Pipeline Genérico de Eventos: Bus desacoplado con templates y lambdas.
Bug Demos: <code>D02_TemplateDeduction</code> , <code>D03_LinkerBug</code> , <code>D07_DanglingLambda</code>		Modelo RAM: Monomorfización en Compilación Decisión: Templates obligatoriamente en <code>.hpp</code>

Módulo 15 — STL Algorithms, Pipelines & Capstone Final		8 Lecciones · 3 Bug Demos
Objetivo: Dominar algoritmos estándar declarativos, Erase-Remove Idiom, iteradores y culminar el Capstone integral del curso.		
L01	Algoritmos STL vs Bucles	Expresividad con <code>std::all_of</code> , <code>any_of</code> y <code>count_if</code> vs <code>range-for</code> .
L02	Invalidación de Iteradores	Prevención de Segfaults mediante el Erase-Remove Idiom y puente a C++20.
L03	Búsqueda y Predicados	<code>std::find_if</code> y <code>std::min_element</code> retornando <code>std::optional</code> .
L04	Transformación y Reducción	Mapeo funcional con <code>std::transform</code> y reducción con <code>std::accumulate</code> .
L05	Ordenamiento Avanzado	<code>std::sort</code> con lambdas y comparadores multicriterio personalizados.
L06	Evolución C++20 Ranges	Evaluación perezosa con tuberías <code> </code> sin vectores intermedios temporales.
L07	Concurrencia Básica	Tareas en segundo plano con <code>std::async</code> y <code>std::future</code> .
L08	CAPSTONE FINAL DEL CURSO	"El Motor RPG Definitivo": Consolidación armónica total de los 15 módulos.
Bug Demos: <code>D01_RawLoopOffByOne</code> , <code>D02_IteratorInvalidation</code> , <code>D07_DataRace</code>		Modelo RAM: Pipelines de Algoritmos Declarativos Decisión: Claridad ante todo: STL para procesar; range-for directo

6. MANDAMIENTOS DE CODIFICACIÓN Y ESTÁNDARES DE INGENIERÍA

1. Veto a "using namespace std;": Usar prefijo explícito <code>std::</code> .	2. Veto a "std::endl": Usar carácter <code>'\n'</code> directo contra cuellos de botella I/O.	3. Inicialización Uniforme {}: Prohibido declarar variables sueltas sin inicializar.
4. Veto a new[] / delete[]: Gestión de memoria exclusiva con <code>std::unique_ptr</code> .	5. Const Correctness: Inmutabilidad por defecto en variables y métodos.	6. Veto a rand() / srand(): Aleatoriedad exclusiva con la librería <code><random></code> .
7. Acceso Verificado .at(): Evitar corchetes <code>[]</code> ciegos en etapas formativas.	8. Destructor Virtual Obligatorio: <code>virtual ~Base() = default;</code> en clases base.	9. Claridad ante todo: Algoritmos STL para procesar; range-for para secuencias directas.

7. BIBLIOGRAFÍA OFICIAL DE REFERENCIA

1. **Bjarne Stroustrup**, *A Tour of C++*, 3rd ed. (C++20), Addison-Wesley Professional, 2022.
2. **Scott Meyers**, *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*, O'Reilly Media, 2014.
3. **Jason Turner**, *C++ Best Practices: 45ish Ways to Write Better Modern C++*, 2nd ed., Leanpub, 2022.
4. **Bjarne Stroustrup & Herb Sutter**, *C++ Core Guidelines*, Standard C++ Foundation (isocpp.github.io/CppCoreGuidelines), 2026.
5. **ISO/IEC JTC1/SC22/WG21**, *Programming Language — C++ (ISO/IEC 14882:2020)*, International Organization for Standardization, 2020.